

Problem Description

The Monk recently learnt about priority queues and requested his teacher for a fun challenge to solve. As a result, his teacher devised a simple task. He now possesses A , which is an integer array. He wishes to discover the product of the greatest, second largest, and third largest integers in the range $[1, i]$ for each index i .

Note: Two numbers can be the same value-wise but they should be distinct index-wise.

Constraints:

$$1 \leq N \leq 100000$$

$$0 \leq A[i] \leq 1000000$$

Input:

The first line contains an integer N , denoting the number of elements in the array A .

The next line contains N space separated integers, each denoting the i th integer of the array A .

Output:

Print the answer for each index in each line. If there is no second largest or third largest number in the array A upto that index, then print "-1", without the quotes.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

5
1 2 3 4 5

EXPECTED OUTPUT

-1
-1
6
24
60

Test Case 2

INPUT (STDIN)

8
3 7 1 2 4 7 5 9

EXPECTED OUTPUT

-1
-1
21
42
84
196
245
441

▼ Mandatory Test Cases

Test Case 1

KEYWORD

if(biggest<big)

Test Case 2

KEYWORD

if(a[i]>biggest)

▼ Complexity Test Cases

Test Case 1

Test Case 2

Test Case 3

CYCLOMATIC COMPLEXITY

10

TOKEN COUNT

300

NLOC

62

```
1 #include <stdio.h>
2
3 void findProduct(long long a[], int n) {
4     for (int i = 0; i < n; i++) {
5         if (i < 2) {
6             printf("-1\n");
7             continue;
8         }
9         long long biggest = -1, big = -1, small = -1;
10        for (int j = 0; j <= i; j++) {
11            if (a[j] > biggest) {
12                small = big;
13                big = biggest;
14                biggest = a[j];
15            } else if (a[j] > big) {
16                small = big;
17                big = a[j];
18            } else if (a[j] > small) {
19                small = a[j];
20            }
21        }
22        if (biggest < big) {
23
24        }
25        printf("%lld\n", biggest * big * small);
```

Ln 39, Col 1 Spaces: 4 UTF-8 CRLF C

```
26     }  
27 }  
28  
29 int main() {  
30     int n;  
31     if (scanf("%d", &n) != 1) return 0;  
32     long long a[n];
```

```
33     for (int i = 0; i < n; i++) {
34         scanf("%lld", &a[i]);
35     }
36     findProduct(a, n);
37     return 0;
38 }
39
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

5

1 2 3 4 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Queue1.exe

-1

-1

6

24

60

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

8

3 7 1 2 4 7 5 9

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Queue1.exe

-1

-1

21

42

84

196

245

441

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

Question description

There is a bit string consisting of n bits. Then, there are some changes that invert one given bit. Your task is to report, after each change, the length of the longest substring whose each bit is the same.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq m \leq 2 \cdot 10^5$
- $1 \leq x_i \leq n$

Input Format

The first input line has a bit string consisting of n bits. The bits are numbered $1, 2, \dots, n$.

The next line contains an integer m : the number of changes.

The last line contains m integers $x_1, x_2, \dots, x_m$ describing the changes.

Output Format

After each change, print the length of the longest substring whose each bit is the same.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

001011
3
3 2 5

EXPECTED OUTPUT

4 2 3

Test Case 2

INPUT (STDIN)

101010
4
1 3 2 6

EXPECTED OUTPUT

2 4 2 2

▼ Mandatory Test Cases

Test Case 1

KEYWORD

void pull(int k,int
l,int r)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

2

Test Case 2

TOKEN COUNT

550

Test Case 3

NLOC

59

```
1 #include <stdio.h>
2 #include <string.h>
3
4 #define MAXN 200005
5
6 char s[MAXN];
7
8 int max(int a, int b) { return a > b ? a : b; }
9
10 int getLongest(int n) {
11     if (n == 0) return 0;
12     int max_len = 1;
13     int cur_len = 1;
14     for (int i = 1; i < n; i++) {
15         if (s[i] == s[i - 1]) {
16             cur_len++;
17         } else {
18             if (cur_len > max_len) max_len = cur_len;
19             cur_len = 1;
20         }
21     }
22     if (cur_len > max_len) max_len = cur_len;
23     return max_len;
24 }
```

```
26 void pull(int k, int l, int r) {
27
28 }
29
30 int main() {
31     if (scanf("%s", s) != 1) return 0;
32     int n = strlen(s);
33     int m;
34     if (scanf("%d", &m) != 1) return 0;
35
36     for (int i = 0; i < m; i++) {
```

```
37     int idx;
38     scanf("%d", &idx);
39     idx--;
40     if (s[idx] == '0') s[idx] = '1';
41     else s[idx] = '0';
42
43     printf("%d%c", getLongest(n), (i == m - 1) ? '\n' : ' ');
44 }
45 return 0;
46}
47
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

001011

3

3 2 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Queue2.exe

4 2 3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

101010

4

1 3 2 6

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Queue2.exe

2 4 2 2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

DS	Session	Queue	Question Information	Level 1	Challenge 53
Question description Ramesh is an DS expert training youngsters struggling in DS to make them better. Ramesh usually gives interesting problems to the youngsters to make them love the DS. One such day Sathya provided to the youngsters to solve the task such that, delete an element in a Queue, Queue data structures work on the FIFO architecture so the element that has entered first in the list will go out from the list first. Youngsters were lacking the idea to solve the problem. Being an exciting youngster can you solve it? Function Description					

1. Create a main function for the program.
2. In this main function accept the size and data of the queue by the user.
3. After that enqueue these element in the queue using an enqueue() function
4. Now to dequeue, we will be using an function that will remove elements from the queue one by one.
5. In this dequeue function first we will check if there are elements present in the queue or not.
6. If there are elements present in the queue then we will move the front of the queue by one step each time ,that is every time when an element is removed from the list.

Constraints:

$0 < \text{size} < 100$

$0 < \text{data} < 1000$

Input format:

First line indicates the size of the queue

Second line indicates the elements of the queue.

Output Format:

every lines indicates the enqueue of each elements

last line indicates the fine queued elements.

Logical Test Cases

Test Case 1

INPUT (STDIN)

5
5 6 3 1 2

EXPECTED OUTPUT

Dequeuing elements:
6 3 1 2
3 1 2
1 2
2

Test Case 2

INPUT (STDIN)

8
1 8 6 4 9 5 3 5

EXPECTED OUTPUT

Dequeuing elements:
8 6 4 9 5 3 5
6 4 9 5 3 5
4 9 5 3 5
9 5 3 5
5 3 5
3 5
5

Mandatory Test Cases

Test Case 1

KEYWORD

void enqueue(int data)

Test Case 2

KEYWORD

for(i=front;i<=rear;i++)

Test Case 3

KEYWORD

enqueue(data);

Test Case 4

KEYWORD

void dequeue()

Complexity Test Cases

Test Case 1

Test Case 2

Test Case 3

CYCLOMATIC COMPLEXITY

3

TOKEN COUNT

250

NLOC

59

```
Queue3.c +
1 #include <stdio.h>
2
3 #define MAX 100
4
5 int queue[MAX];
6 int front = -1;
7 int rear = -1;
8
9 void enqueue(int data) {
10     if (rear == MAX - 1) return;
11     if (front == -1) front = 0;
12     queue[++rear] = data;
13 }
14
15 void dequeue() {
16     if (front == -1 || front > rear) return;
17     front++;
18 }
19
20 void display() {
21     int i;
22     for (i = front; i <= rear; i++) {
23
24     }
25 }
```

```
26
27int main() {
28    int size;
29    if (scanf("%d", &size) != 1) return 0;
30
31    for (int i = 0; i < size; i++) {
32        int data;
33        scanf("%d", &data);
34        enqueue(data);
35    }
```

```
36
37     printf("Dequeuing elements:\n");
38     while (front < rear) {
39         dequeue();
40         display();
41     }
42     return 0;
43 }
44
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

5

5 6 3 1 2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Queue3.exe

Dequeuing elements:

6 3 1 2

3 1 2

1 2

2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

8

1 8 6 4 9 5 3 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Queue3.exe

Dequeuing elements:

8 6 4 9 5 3 5

6 4 9 5 3 5

4 9 5 3 5

9 5 3 5

5 3 5

3 5

5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

Question description

Joe root is a Placement trainer. he is working as CDC trainer in reputed institution that during training the youngsters are struggling in queue concept.

Joe root usually gives interesting problems to the students to make them love the DS.

One such day Joe root provided to the final year students to solve the task such that, Queue implementation with arrays as using linked list for implementing queue and delete an element from the queue using linked list concept.

Queue data structures work on the FIFO architecture so the element that has entered first in the list will go out from the list first.

Final Year students were lacking the idea to solve the problem.

Being an exciting youngster can you solve it?

Function Description

enqueue(data)

- Build a new node with given data.
- Check if the queue is empty or not.
- If queue is empty then assign new node to front and rear.
- Else make next of rear as new node and rear as new node.

dequeue()

- Check if queue is empty or not.
- If queue is empty then dequeue is not possible.
- Else store front in temp
- And make next of front as front.

print()

- Check if there is some data in the queue or not.
- If the queue is empty print “No data in the queue.”
- Else define a node pointer and initialize it with front.
- Print data of node pointer until the next of node pointer becomes NULL.

Constraints:

0<size <100

0<data<1000

Input format:

First line indicates the size of the queue

Second line indicates the elements of the queue.

Output Format:

First line indicates the inserted elements in queue using linked list concept

second line indicates the one element dequeued from the queue using linked list concept.

Logical Test Cases

Test Case 1

INPUT (STDIN)

5
9 1 5 3 5

EXPECTED OUTPUT

9 1 5 3 5
1 5 3 5

Test Case 2

INPUT (STDIN)

8
1 8 6 4 9 5 3 5

EXPECTED OUTPUT

1 8 6 4 9 5 3 5
8 6 4 9 5 3 5

Mandatory Test Cases

Test Case 1

KEYWORD

void enqueue(int d)

Test Case 2

KEYWORD

struct node* new_n;

Test Case 3

KEYWORD

new_n = (struct node*)malloc(sizeof(struct node));

Test Case 4

KEYWORD

struct node *rear = NULL;

Test Case 5

KEYWORD

struct node *front = NULL;

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

3

Test Case 2

TOKEN COUNT

300

Test Case 3

NLOC

70

Queue4.c +

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct node {
5     int data;
6     struct node* next;
7 };
8
9 struct node *front = NULL;
10 struct node *rear = NULL;
11
12 void enqueue(int d) {
13     struct node* new_node = (struct node*)malloc(sizeof(struct node));
14     new_node->data = d;
15     new_node->next = NULL;
16     if (front == NULL && rear == NULL) {
17         front = rear = new_node;
18         return;
19     }
20     rear->next = new_node;
21     rear = new_node;
22 }
23
24 void dequeue() {
25     if (front == NULL) return;
```

```
26 struct node* temp = front;
27 front = front->next;
28 if (front == NULL) rear = NULL;
29 free(temp);
30}
```

main | CH.SC.U4CSE25225 | Queue4.c

Ln 59, Col 1 Spaces: 4 UTF-8 CRLF C

```
31
32void print() {
33    if (front == NULL) {
34        printf("No data in the queue.\n");
35        return;
36    }
37    struct node* temp = front;
```

```
38     while (temp != NULL) {
39         printf("%d%c", temp->data, (temp->next == NULL) ? '\n' : ' ');
40         temp = temp->next;
41     }
42 }
43
44 int main() {
45     int size;
46     if (scanf("%d", &size) != 1) return 0;
47
48     for (int i = 0; i < size; i++) {
```

```
49     int d;
50     scanf("%d", &d);
51     enqueue(d);
52 }
53
54 print();
55 dequeue();
56 print();
57 return 0;
58}
59
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

5

9 1 5 3 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Queue4.exe

9 1 5 3 5

1 5 3 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

8

1 8 6 4 9 5 3 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Queue4.exe

1 8 6 4 9 5 3 5

8 6 4 9 5 3 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

DS	Session	Queue	Question Information	Level 1	Challenge 55
Question description Sathya is an DS expert training youngsters struggling in DS to make them better. Sathya usually gives interesting problems to the youngsters to make them love the DS. One such day Sathya provided to the youngsters to solve the task such that, insert an element in a Queue in FIFO order Youngsters were lacking the idea to solve the problem. Being an exciting youngster can you solve it? Function Description					
CH.SC.U4CSE25225 - M ASHWIN Department of Computer Science & Engineering (CSE) 2026					

1. Define the maximum size of queue and initialize front and rear as -1.
2. In the main function we will initialize two variables that will store the data and the size of the queue.
3. Accept the data that we want to enter in a queue using a for loop.
4. After accepting the data use enqueue() function to insert the data in a queue.
5. In this function return queue id full , if the value of rear is equal to max-1.
6. Else increase the value of rear by 1.
7. After this insert the data that we have accepted earlier.
8. Now display the data of the queue using a disp() function.
9. In this function if the value of front is equal to -1 then it means list is empty.
10. Else initialize a for loop from front to rear and print the data that queue is holding.

Constraints:

$0 < \text{size} < 100$

$0 < \text{data} < 1000$

Input format:

First line indicates the size of the queue

Second line indicates the elements of the queue.

Output Format:

every lines indicates the enqueue of each elements

last line indicates the fine queued elements.

Logical Test Cases

Test Case 1

INPUT (STDIN)

5
5 6 3 1 2

EXPECTED OUTPUT

Enqueueing 5
5 Enqueueing 6
5 6 Enqueueing 3
5 6 3 Enqueueing 1
5 6 3 1 Enqueueing 2
5 6 3 1 2

Test Case 2

INPUT (STDIN)

8
1 8 6 4 9 5 3 5

EXPECTED OUTPUT

Enqueueing 1
1 Enqueueing 8
1 8 Enqueueing 6
1 8 6 Enqueueing 4
1 8 6 4 Enqueueing 9
1 8 6 4 9 Enqueueing 5
1 8 6 4 9 5 Enqueueing 3
1 8 6 4 9 5 3 Enqueueing 5
1 8 6 4 9 5 3 5

Mandatory Test Cases

Test Case 1

KEYWORD

void enqueue(int data)

Test Case 2

KEYWORD

for(i=front;i<=rear;i++)

Test Case 3

KEYWORD

enqueue(data);

Complexity Test Cases

Test Case 1

Test Case 2

Test Case 3

CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
3	200	44

```
Queue5.c +
1 #include <stdio.h>
2
3 #define MAX 100
4
5 int queue[MAX];
6 int front = -1;
7 int rear = -1;
8
9 void enqueue(int data) {
10     if (rear == MAX - 1) return;
11     if (front == -1) front = 0;
12     queue[++rear] = data;
13 }
14
15 void disp() {
16     int i;
17     for (i = front; i <= rear; i++) {
18         printf("%d ", queue[i]);
19     }
20 }
21
22 int main() {
23     int size;
24     if (scanf("%d", &size) != 1) return 0;
25
```

```
26  for (int i = 0; i < size; i++) {  
27      int data;  
28      scanf("%d", &data);  
29      if (i > 0) {  
30          disp();  
31      }  
32      printf("Enqueuing %d\n", data);
```

```
33     enqueue(data);
34 }
35 disp();
36 printf("\n");
37 return 0;
38}
39
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

5

5 6 3 1 2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Queue5.exe

Enqueuing 5

5 Enqueuing 6

5 6 Enqueuing 3

5 6 3 Enqueuing 1

5 6 3 1 Enqueuing 2

5 6 3 1 2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

8

1 8 6 4 9 5 3 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Queue5.exe

Enqueuing 1

1 Enqueuing 8

1 8 Enqueuing 6

1 8 6 Enqueuing 4

1 8 6 4 Enqueuing 9

1 8 6 4 9 Enqueuing 5

1 8 6 4 9 5 Enqueuing 3

1 8 6 4 9 5 3 Enqueuing 5

1 8 6 4 9 5 3 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

DS	Session	Queue	Question Information	Level 1	Challenge 56
----	---------	-------	----------------------	---------	--------------

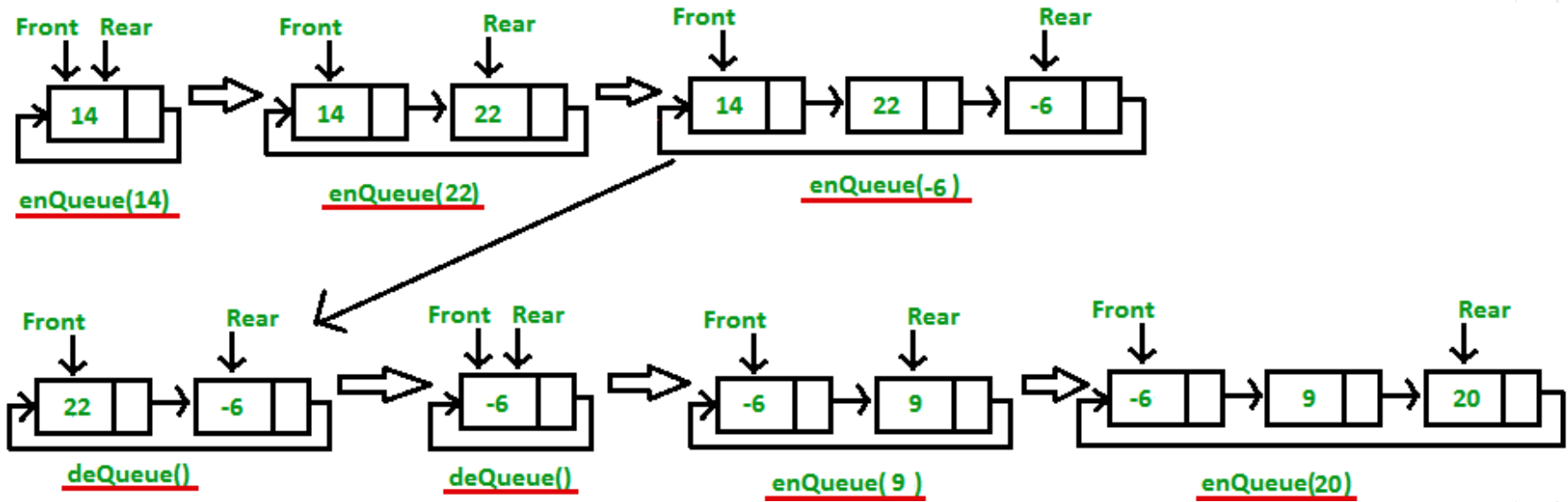
Question description

Lalitha is a B.Tech student. During her final year Campus Interview, she has an opportunity to get a job in a software company in Bangalore.

The company provides Five months training period with Rs.30000/month Package. Then it will be incremented to Rs.55000 per month.

At the end of the training, the examination was conducted for all freshers, Lalitha got a question paper and one of the questions comes under the concept of Queues in data structure that is Circular Linked List Implementation in Circular Queue.

Can you help?



Constraints

First 6 elements inserted in array 0, next 6 elements inserted into array 1, remaining elements inserted into array 2.

Input Format

First line indicates the number of elements to be inserted in the queues.

Second line indicates the elements.

Output Format

first line indicates the dequeue element of Q2

Second line indicates the dequeue element of Q1

Third line line indicates the dequeue element of Q0

Note: Refer sample input and output test cases

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
10
1 2 3 4 5 6 7 8 9 10
```

EXPECTED OUTPUT

```
Elements in Circular Queue are:1 2 3 4 5 6 7 8 9 10
Deleted value = 1
Deleted value = 2Elements in Circular Queue are:3 4
5 6 7 8 9 10
```

Test Case 2

INPUT (STDIN)

```
10
16 17 18 19 20 11 12 13 14 15
```

EXPECTED OUTPUT

```
Elements in Circular Queue are:16 17 18 19 20 11 12
13 14 15
Deleted value = 16
Deleted value = 17Elements in Circular Queue are:18
19 20 11 12 13 14 15
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void enqueue(Queue* q,int value)
```

Test Case 2

KEYWORD

```
int dequeue(Queue* q)
```

Test Case 3

KEYWORD

```
void displayQueue(struct Queue*
```

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

2

Test Case 2

TOKEN COUNT

410

Test Case 3

NLOC

74

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node* link;
7 };
8
9 typedef struct Queue {
10     struct Node *front, *rear;
11 } Queue;
12
13 void enqueue(Queue* q, int value) {
14     struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
15     temp->data = value;
16     temp->link = NULL;
17     if (q->front == NULL) {
18         q->front = temp;
19     } else {
20         q->rear->link = temp;
21     }
22     q->rear = temp;
23     q->rear->link = q->front;
24 }
25
```

```

26 int deQueue(Queue* q) {
27     if (q->front == NULL) return -1;
28     int value;
29     if (q->front == q->rear) {
30         value = q->front->data;
31         free(q->front);
32         q->front = NULL;
33         q->rear = NULL;
34     } else {
35         struct Node* temp = q->front;
36         value = temp->data;
37         q->front = q->front->link;
38         q->rear->link = q->front;
39         free(temp);
40     }
41     return value;
42 }

```

main | CH.SC.U4CSE25225 | Queue6.c

Ln 75, Col 1 Spaces: 4 UTF-8 CRLF C

```
43
44void displayQueue(struct Queue* q) {
45    struct Node* temp = q->front;
46    printf("Elements in Circular Queue are:");
47    if (temp != NULL) {
48        do {
49            printf("%d ", temp->data);
50            temp = temp->link;
51        } while (temp != q->front);
52    }
53    printf("\n");
54}
55
56int main() {
57    int n;
58    if (scanf("%d", &n) != 1) return 0;
59
```

```
60 Queue* q = (Queue*)malloc(sizeof(Queue));
61 q->front = q->rear = NULL;
62
63 for (int i = 0; i < n; i++) {
64     int val;
65     scanf("%d", &val);
66     enqueue(q, val);
67 }
68
69 displayQueue(q);
70 printf("Deleted value = %d\n", dequeue(q));
71 printf("Deleted value = %d\n", dequeue(q));
72 displayQueue(q);
73 return 0;
74}
75
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
10
```

```
1 2 3 4 5 6 7 8 9 10
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Queue6.exe
```

```
Elements in Circular Queue are:1 2 3 4 5 6 7 8 9 10
```

```
Deleted value = 1
```

```
Deleted value = 2
```

```
Elements in Circular Queue are:3 4 5 6 7 8 9 10
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
10
```

```
16 17 18 19 20 11 12 13 14 15
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Queue6.exe
```

```
Elements in Circular Queue are:16 17 18 19 20 11 12 13 14 15
```

```
Deleted value = 16
```

```
Deleted value = 17
```

```
Elements in Circular Queue are:18 19 20 11 12 13 14 15
```

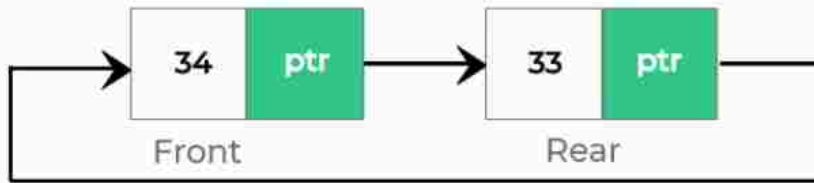
```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

DS	Session	Queue	Question Information	Level 1	Challenge 57
Question description					
Anderson is a Placement trainer. he is working as CDC trainer in reputed institution that during training the youngsters are struggling in queue concept.					
Anderson usually gives interesting problems to the students to make them love the DS.					
One such day Joe Anderson provided to the final year students to solve the task such that, Circular Queue using Linked List. there is no memory waste while using Circular Queue, it is preferable than using a regular queue.					
Because linked lists allow for dynamic memory allocation, they are simple to build.					
Circular Queue implementation using linked list is identical to circular linked list except that circular Queue has two pointers front and back whereas circular linked list only has one pointer head.					
Final Year students were lacking the idea to solve the problem.					
Being an exciting youngster can you solve it?					
Function Description					

enqueue(34)



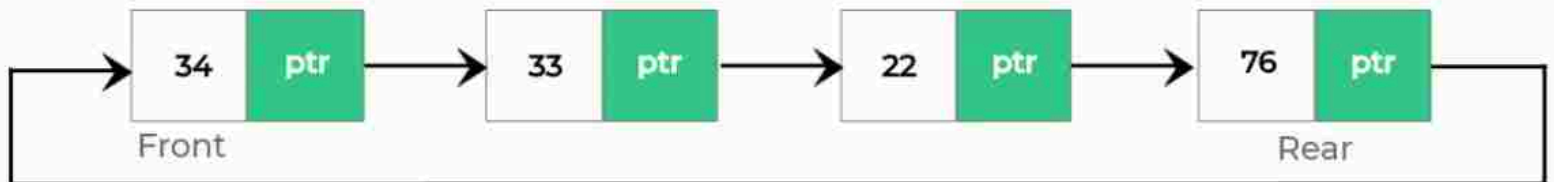
enqueue(33)



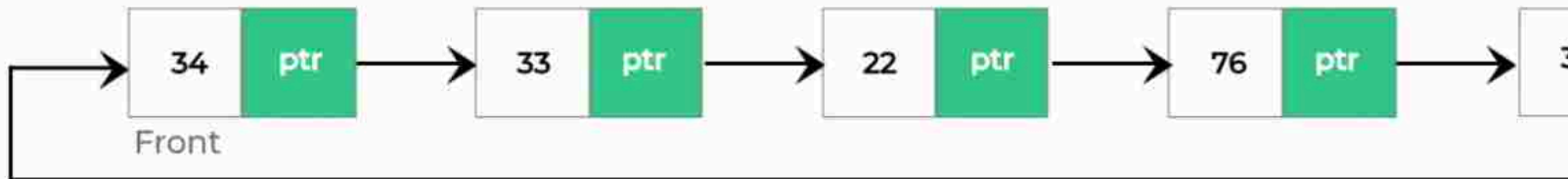
enqueue(22)



enqueue(76)

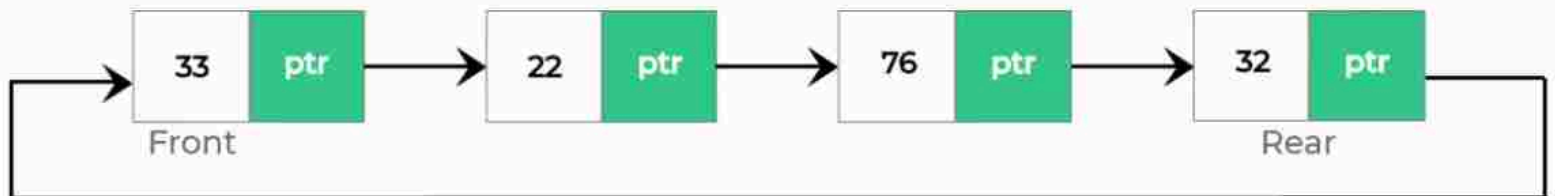


enqueue(32)



Removing the elements from Queue

dequeue()



Constraints:

0 < size < 100

0 < data < 1000

Input format:

First line indicates the size of the queue

Second line indicates the elements of the queue.

Output Format:

First line indicates the inserted elements in queue using linked list concept

second line indicates the one element dequeued from the queue using linked list concept.

Logical Test Cases

Test Case 1

INPUT (STDIN)

5
9 1 5 3 5

EXPECTED OUTPUT

9
1
5
3
5

Test Case 2

INPUT (STDIN)

8
1 8 6 4 9 5 3 5

EXPECTED OUTPUT

1
8
6
4
9
5
3
5

Mandatory Test Cases

Test Case 1

KEYWORD

void enqueue(int d)

Test Case 2

KEYWORD

struct node* next;

Test Case 3

KEYWORD

n = (struct node*)malloc(sizeof(struct n

Test Case 4

KEYWORD

struct node *f = NULL;

Test Case 5

KEYWORD

struct node *r = NULL;

Test Case 6

KEYWORD

n->data = d;

Test Case 7

KEYWORD

n->next = NULL;

Test Case 8

KEYWORD

struct node* t;

Complexity Test Cases

Test Case 1

Test Case 2

Test Case 3

CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
3	330	72
CH.SC.U4CSE25225 - M ASHWIN Department of Computer Science & Engineering (CSE) 2026		

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct node {
5     int data;
6     struct node* next;
7 };
8
9 struct node *f = NULL;
10 struct node *r = NULL;
11
12 void enqueue(int d) {
13     struct node* n = (struct node*)malloc(sizeof(struct node));
14     n->data = d;
15     n->next = NULL;
16     if (f == NULL && r == NULL) {
17         f = r = n;
18         r->next = f;
19     } else {
20         r->next = n;
21         r = n;
22         r->next = f;
23     }
24 }
25
```

```
26 void dequeue() {
27     struct node* t = f;
28     if (f == NULL && r == NULL) {
29         return;
30     } else if (f == r) {
31         f = r = NULL;
32         free(t);
33     } else {
34         f = f->next;
35         t->next = t;
36         free(t);
37     }
38 }
39
```

main | CH.SC.U4CSE25225 | Queue7.c

Ln 67, Col 1 Spaces: 4 UTF-8 CRLF C

```
40 void display() {
41     struct node* t = f;
42     if (f == NULL && r == NULL) {
43         return;
44     }
45     do {
46         printf("%d ", t->data);
47         t = t->next;
48     } while (t != f);
49     printf("\n");
50 }
51
52 int main() {
53     int size;
```

```
54     if (scanf("%d", &size) != 1) return 0;
55
56     for (int i = 0; i < size; i++) {
57         int d;
58         scanf("%d", &d);
59         enqueue(d);
60     }
61
62     display();
63     dequeue();
64     display();
65     return 0;
66 }
67
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

5

9 1 5 3 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Queue7.exe

9 1 5 3 5

1 5 3 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

8

1 8 6 4 9 5 3 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Queue7.exe

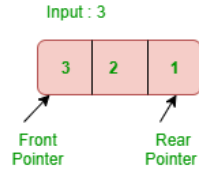
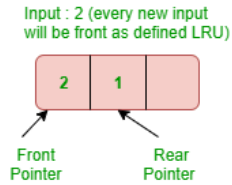
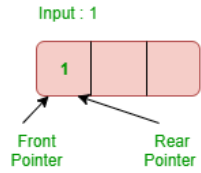
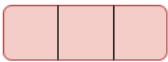
1 8 6 4 9 5 3 5

8 6 4 9 5 3 5

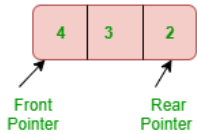
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

DS	Session	Queue	Question Information	Level 1	Challenge 58
Question description Selvan is very interested in surfing the contents from google. He searches for various coding test on Google. One day he searched about online coding competitions, in the retrieval links, he received many links for coding competition. he chooses first link from the goole suggestion list. first question for the coding competition is LRU cache implementation using queue concepts. Function Description 1. Queue which is implemented using a doubly linked list. The maximum size of the queue will be equal to the total number of frames available (cache size). The most recently used pages will be near front end and least recently pages will be near the rear end. 2. A Hash with page number as key and address of the corresponding queue node as value.					
CH.SC.U4CSE25225 - M ASHWIN Department of Computer Science & Engineering (CSE) 2026					

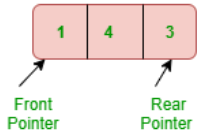
Given 3 page frames, so we take size of Queue is 3. Initially, Queue is empty.



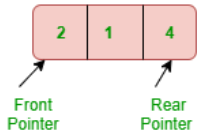
Input : 4 (since all the frames are full, we remove a node from the rear of queue, and add the new node to the front of queue.)



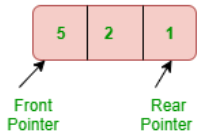
Input : 1 (since all the frames are full, we remove a node from the rear of queue, and add the new node to the front of queue.)



Input : 2 (since all the frames are full, we remove a node from the rear of queue, and add the new node to the front of queue.)



Input : 5 (since all the frames are full, we remove a node from the rear of queue, and add the new node to the front of queue.)



Constraints

For this experiment, cache can hold 4 pages.

Let 10 different pages can be requested (pages to be referenced are numbered from 0 to 9).

Input Format:

First line represents n and m, where n is the page number with in the range (0-9) & m is cache size (must be 4 for this problem).

Next line represents the reference pages.

Output Format:

Single line represents the cache frames after the above referenced pages.

Test Case 1

INPUT (STDIN)

```
6 4
1 2 3 1 4 5
```

EXPECTED OUTPUT

```
5 4 1 3
```

Test Case 2

INPUT (STDIN)

```
10 4
3 1 4 5 9 8 7 6 1 2
```

EXPECTED OUTPUT

```
2 1 6 7
```

▼ Mandatory Test Cases

Test Case 1

KEYWORD

```
QNode* newNode(unsigned pageNumber)
```

Test Case 2

KEYWORD

```
Queue* createQueue(int numberOfFrames)
```

Test Case 3

KEYWORD

```
typedef struct Queue
```

Test Case 4

KEYWORD

```
typedef struct QNode
```

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
1
```

Test Case 2

TOKEN COUNT

```
750
```

Test Case 3

NLOC

```
120
```

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct QNode {
5     unsigned pageNumber;
6     struct QNode *prev, *next;
7 } QNode;
8
9 typedef struct Queue {
10     unsigned count;
11     unsigned numberOfFrames;
12     QNode *front, *rear;
13 } Queue;
14
15 QNode* newQNode(unsigned pageNumber) {
16     QNode* temp = (QNode*)malloc(sizeof(QNode));
17     temp->pageNumber = pageNumber;
18     temp->prev = temp->next = NULL;
19     return temp;
20 }
21
22 Queue* createQueue(int numberOfFrames) {
23     Queue* queue = (Queue*)malloc(sizeof(Queue));
24     queue->count = 0;
25     queue->front = queue->rear = NULL;
```

```

26 queue->numberOfFrames = numberOfFrames;
27 return queue;
28}
29
30void removeNode(Queue* queue, QNode* node) {
31     if (node->prev != NULL) {
32         node->prev->next = node->next;
33     } else {
34         queue->front = node->next;
35     }
36     if (node->next != NULL) {
37         node->next->prev = node->prev;
38     } else {
39         queue->rear = node->prev;
40     }
41     queue->count--;
42}
43
44void addToFront(Queue* queue, QNode* node) {

```

```

45     node->next = queue->front;
46     node->prev = NULL;
47     if (queue->front != NULL) {
48         queue->front->prev = node;
49     }
50     queue->front = node;
51     if (queue->rear == NULL) {
52         queue->rear = node;
53     }
54     queue->count++;
55 }
56
57 void referencePage(Queue* queue, unsigned pageNumber) {
58     QNode* curr = queue->front;
59     QNode* found = NULL;
60     while (curr != NULL) {
61         if (curr->pageNumber == pageNumber) {
62             found = curr;
63             break;

```

```

64     }
65     curr = curr->next;
66 }
67
68 if (found != NULL) {
69     removeNode(queue, found);
70     addToFront(queue, found);
71 } else {
72     if (queue->count == queue->numberOfFrames) {
73         QNode* lru = queue->rear;
74         removeNode(queue, lru);
75         free(lru);
76     }
77     QNode* newNode = newQNode(pageNumber);
78     addToFront(queue, newNode);
79 }
80}
81
82int main() {

```

```

83  int n, k;
84  if (scanf("%d %d", &n, &k) != 2) return 0;
85
86  Queue* queue = createQueue(k);
87
88  for (int i = 0; i < n; i++) {
89      unsigned page;
90      scanf("%u", &page);
91      referencePage(queue, page);
92  }
93
94  QNode* curr = queue->front;
95  while (curr != NULL) {
96      printf("%u%c", curr->pageNumber, (curr->next == NULL) ? '\n' : ' ');
97      curr = curr->next;
98  }
99  return 0;
100
101

```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

6 4

1 2 3 1 4 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Queue8.exe

5 4 1 3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

10 4

3 1 4 5 9 8 7 6 1 2

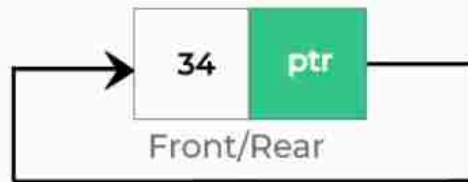
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Queue8.exe

2 1 6 7

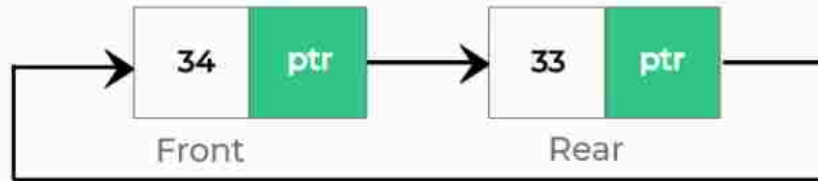
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

DS	Session	Queue	Question Information	Level 1	Challenge 59
Question description On the last day of the semester, Shahid's students were talking and playing very loudly to the delight of the end of the semester. The principal of the college severely reprimanded Shahid. But he decided to engage them in a different activity without getting angry at the students. So Shahid gave his students to solve the task such that, Circular Queue using Linked List and dequeue two elements from circular queue. there is no memory waste while using Circular Queue, it is preferable than using a regular queue. Because linked lists allow for dynamic memory allocation, they are simple to build. Circular Queue implementation using linked list is identical to circular linked list except that circular Queue has two pointers front and back whereas circular linked list only has one pointer head. Final Year students were lacking the idea to solve the problem. Being an exciting youngster can you solve it? Function Description					
CH.SC.U4CSE25225 - M ASHWIN Department of Computer Science & Engineering (CSE) 2026					

enqueue(34)



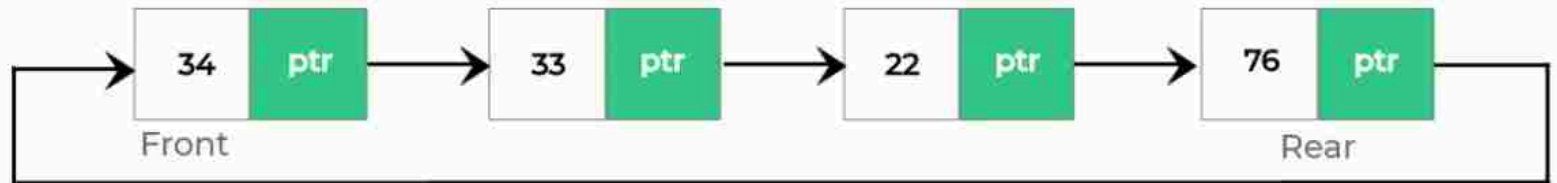
enqueue(33)



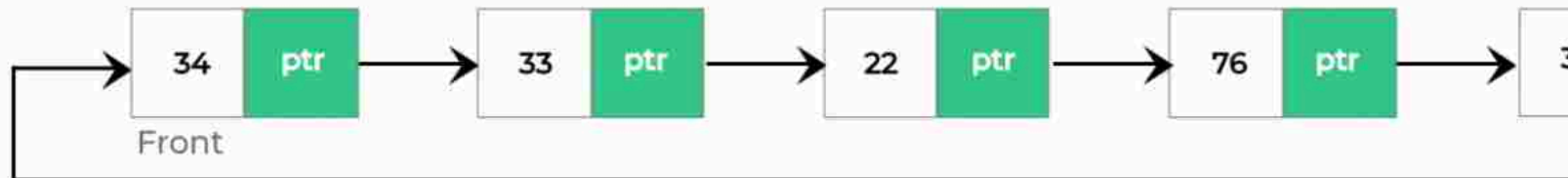
enqueue(22)



enqueue(76)

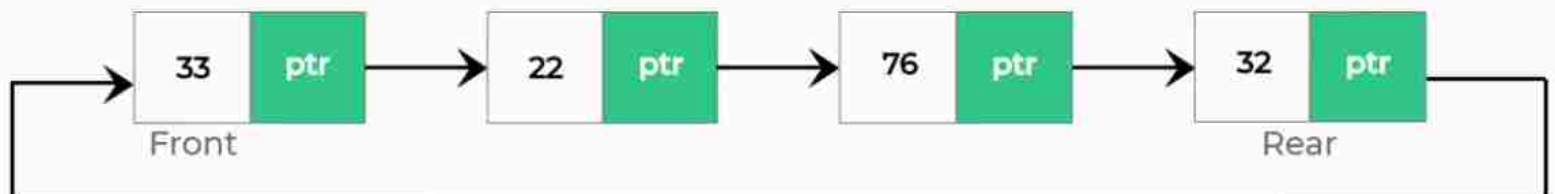


enqueue(32)



Removing the elements from Queue

dequeue()



Constraints:

0 < size < 100

0 < data < 1000

Input format:

First line indicates the size of the queue

Second line indicates the elements of the queue.

Output Format:

First line indicates the inserted elements in queue using linked list concept

second line indicates the one element dequeued from the queue using linked list concept.

Logical Test Cases

Test Case 1

INPUT (STDIN)

5
9 1 5 3 5

EXPECTED OUTPUT

9 1 5 3 5
1 5 3 5
5 3 5

Test Case 2

INPUT (STDIN)

8
1 8 6 4 9 5 3 5

EXPECTED OUTPUT

1 8 6 4 9 5 3 5
8 6 4 9 5 3 5
6 4 9 5 3 5

Mandatory Test Cases

Test Case 1

KEYWORD

void enqueue(int d)

Test Case 2

KEYWORD

struct node* next;

Test Case 3

KEYWORD

n = (struct node*)malloc(sizeof(struct n

Test Case 4

KEYWORD

struct node *f = NULL;

Test Case 5

KEYWORD

struct node *r = NULL;

Test Case 6

KEYWORD

n->data = d;

Test Case 7

KEYWORD

n->next = NULL;

Test Case 8

KEYWORD

struct node* t;

Complexity Test Cases

Test Case 1

CYCIOMATIC COMPLEXITY

3

Test Case 2

TOKEN COUNT

355

Test Case 3

NLOC

75

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct node {
5     int data;
6     struct node* next;
7 };
8
9 struct node *f = NULL;
10 struct node *r = NULL;
11
12 void enqueue(int d) {
13     struct node* n = (struct node*)malloc(sizeof(struct node));
14     n->data = d;
15     n->next = NULL;
16     if (f == NULL && r == NULL) {
17         f = r = n;
18         r->next = f;
19     } else {
20         r->next = n;
21         r = n;
22         r->next = f;
23     }
24 }
25
```

```
26void dequeue() {
27    struct node* t = f;
28    if (f == NULL && r == NULL) {
29        return;
30    } else if (f == r) {
31        f = r = NULL;
32        free(t);
33    } else {
34        f = f->next;
35        r->next = f;
```

main | CH.SC.U4CSE25225 | Queue9.c

Ln 69, Col 1 Spaces: 4 UTF-8 CRLF C

```
36    free(t);
37    }
38}
39
40void display() {
```

```
41     struct node* t = f;
42     if (f == NULL && r == NULL) {
43         return;
44     }
45     do {
46         printf("%d ", t->data);
47         t = t->next;
48     } while (t != f);
49     printf("\n");
50 }
51
52 int main() {
53     int size;
54     if (scanf("%d", &size) != 1) return 0;
55
```

```
56     for (int i = 0; i < size; i++) {  
57         int d;  
58         scanf("%d", &d);  
59         enqueue(d);  
60     }  
61  
62     display();  
63     dequeue();  
64     display();  
65     dequeue();  
66     display();  
67     return 0;  
68 }  
69
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

5

9 1 5 3 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Queue9.exe

9 1 5 3 5

1 5 3 5

5 3 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

8

1 8 6 4 9 5 3 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Queue9.exe

1 8 6 4 9 5 3 5

8 6 4 9 5 3 5

6 4 9 5 3 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

DS	Session	Queue	Question Information	Level 1	Challenge 60
Question description					
Umesh is an DS expert training youngsters struggling in DS to make them better.					
Umesh usually gives interesting problems to the youngsters to make them love the DS.					
One such day Umesh provided to the youngsters to solve the task such that, Reverse a Queue, Queue data structures work on the FIFO architecture so the element that has entered first in the list will go out from the list first.					
Youngsters were lacking the idea to solve the problem.					
Being an exciting youngster can you solve it?					
Function Description					
• INITIALIZE FRONT AND REAR = -1 • CALL ENQUEUE FUNCTION WHILE(I<GIVEN SIZE OF QUEUE) • CALL REVERSE • MAKE A FOR LOOP(I=FRONT,J=REAR;I<J;I++;J++) • SWAP(FRONT,REAR)					
Constraints:					
0<size <100					
0<data<1000					
Input format:					
First line indicates the size of the queue					
Second line indicates the elements of the queue.					
Output Format:					
first line indicates the enqueue of each elements					
last line indicates the reversed queue elements.					

Test Case 1

INPUT (STDIN)

5
5 6 3 1 2

EXPECTED OUTPUT

Queue:5 6 3 1 2
Reversed Queue:2 1 3 6 5

Test Case 2

INPUT (STDIN)

8
1 8 6 4 9 5 3 5

EXPECTED OUTPUT

Queue:1 8 6 4 9 5 3 5
Reversed Queue:5 3 5 9 4 6 8 1

▼ Mandatory Test Cases

Test Case 1

KEYWORD

`void enqueue(int data,int l)`

Test Case 2

KEYWORD

`void reverse()`

Test Case 3

KEYWORD

`enqueue(t,n);`

Test Case 4

KEYWORD

`reverse();`

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

4

Test Case 2

TOKEN COUNT

275

Test Case 3

NLOC

57

Queue10.c +

```
1 #include <stdio.h>
2
3 #define MAX 100
4
5 int queue[MAX];
6 int front = -1;
7 int rear = -1;
8
9 void enqueue(int data, int n) {
10     if (rear == MAX - 1) return;
11     if (front == -1) front = 0;
12     queue[++rear] = data;
13 }
14
15 void reverse() {
16     int i = front, j = rear;
17     while (i < j) {
18         int temp = queue[i];
19         queue[i] = queue[j];
20         queue[j] = temp;
21         i++;
22         j--;
23     }
24 }
25
```

```
26 int main() {  
27     int n;  
28     if (scanf("%d", &n) != 1) return 0;  
29  
30     for (int i = 0; i < n; i++) {  
31         int t;  
32         scanf("%d", &t);  
33         enqueue(t, n);  
34     }  
35  
36     printf("Queue:");  
37     for (int i = front; i <= rear; i++) {
```

```
38     printf("%d%c", queue[i], (i == rear) ? '\n' : ' ');
39 }
40
41 reverse();
42
43 printf("Reversed Queue:");
44 for (int i = front; i <= rear; i++) {
45     printf("%d%c", queue[i], (i == rear) ? '\n' : ' ');
46 }
47 return 0;
48}
49
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

5

5 6 3 1 2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Queue10.exe

Queue:5 6 3 1 2

Reversed Queue:2 1 3 6 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

8

1 8 6 4 9 5 3 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Queue10.exe

Queue:1 8 6 4 9 5 3 5

Reversed Queue:5 3 5 9 4 6 8 1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>